



PUBLIC AUDIT RELEASE

PuppyRaffle Security Review

by fuzz755

Table of Contents

About fuzz755

Disclaimer

Risk Classification

Protocol Summary

Executive Summary

Scope

Findings

Findings Count

Findings Summary

High Findings

[H-1] When a player asks for a refund, the `totalAmountCollected` variable is not reduced, causing contract insolvency

[H-2] Refunding a user zeroes its address in the `players` array, causing a revert on all new entries when two or more users have refunded

[H-3] The `withdrawFees` function may always revert due to integer truncation dust, permanently locking protocol fees

[H-4] The `refund` function is not following the Checks-Effects-Interactions pattern, leading to reentrancy vulnerability

[H-6] Overflow can cause a permanent loss of fees

Medium Findings

[M-1] Poor RNG implementation can be exploited by attackers to choose the winner

[M-2] The total amount of player inside a raffle is limited

Low Findings

[L-1] The amount of players that can enter the raffle simultaneously is limited

Informational Findings

[I-1] The `\_isActivePlayer` internal function is never used

About fuzz755

Fuzz755 is an independent smart contracts security researcher. Having found numerous security vulnerabilities in various protocols, he does his best to contribute to the blockchain ecosystem and its protocols by putting time and effort into security research & reviews. Check previous work here or reach out on Twitter @Fuzz755.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where I try to find as many vulnerabilities as possible. I cannot guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	High	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High:** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium:** leads to a moderate material loss of assets in the protocol or moderately harms a group of users.
- **Low:** leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High:** attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium:** only a conditionally incentivized attack vector, but still relatively likely.

- **Low:** has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Protocol Summary

PuppyRaffle is a raffle protocol where participants can enter to win a cute dog NFT. The protocol allows users to enter by paying an entrance fee, and a winner is selected pseudo-randomly after the raffle duration elapses.

Roles

- **Owner:** Deployer of the protocol. Has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.
- **Player:** Participant of the raffle. Can enter the raffle with the `enterRaffle` function and request a refund through the `refund` function.

Executive Summary

A time-boxed security review of the PuppyRaffle repository was conducted by **fuzz755**, from 2026-03-01 — 2026-03-03. A total of **9** issues were uncovered.

Scope

Protocol	PuppyRaffle
Protocol Type	Raffle / NFT
Audit Date	2026-03-01 — 2026-03-03
Audit Version	1.0
Repository	https://github.com/Cyfrin/4-puppy-raffle-audit
Review Commit Hash	df177a5c9808...

In-Scope Files

```
./src/PuppyRaffle.sol
```

Attack vectors covered

Reentrancy

hello world

RNG

hello world

Findings

Findings Count

Severity	Count
High	5
Medium	2
Low	1
Informational	1
Total	9

Findings Summary

ID	Title	Severity	Status
[H-1]	When a player asks for a refund, the `totalAmountCollected` variable is not reduced, causing contract insolvency	High	Acknowledged
[H-2]	Refunding a user zeroes its address in the `players` array, causing a revert on all new entries when two or more users have refunded	High	Fixed
[H-3]	The `withdrawFees` function may always revert due to integer truncation dust, permanently locking protocol fees	High	Fixed
[H-4]	The `refund` function is not following the Checks-Effects-Interactions pattern, leading to reentrancy vulnerability	High	Fixed
[H-6]	Overflow can cause a permanent loss of fees	High	Fixed
[M-1]	Poor RNG implementation can be exploited by attackers to choose the winner	Medium	Pending
[M-2]	The total amount of player inside a raffle is limited	Medium	Fixed
[L-1]	The amount of players that can enter the raffle simultaneously is limited	Low	Fixed

ID	Title	Severity	Status
[I-1]	The `_isActivePlayer` internal function is never used	Informational	Acknowledged

High Findings

[H-1] When a player asks for a refund, the `totalAmountCollected` variable is not reduced, causing contract insolvency

HIGH

Acknowledged

DESCRIPTION

When a player calls the `refund` function to get his money back, the `players` array's size stays the same. However, the prize pool is calculating with this formula:

```
totalAmountCollected = players.length * entranceFee
```

In reality, the contract balance is only $(\text{players.length} - 1) * \text{entranceFee}$ because of the refund. Now if we call the `selectWinner` function, the transaction will revert on the call:

```
(bool success,) = winner.call{value: prizePool}("");  
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

This does mean that the funds are permanently locked in the contract. The only way to recover them is to send some Ether to the contract, to make its balance greater than or equal to the prize pool.

IMPACT

Funds can be permanently locked in the contract, making the protocol insolvent after any refund.

PROOF OF CONCEPT

Actors:

- Players: Normal players entering the raffle.

```
function test_RaffleBurnPrize() public {  
    // Four addresses are entering the raffle  
    address[] memory players = new address[](4);  
    players[0] = playerOne;  
    players[1] = playerTwo;  
    players[2] = playerThree;  
    players[3] = playerFour;  
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);  
}
```

```

// First player is refunding his ticket
uint256 index0fPlayerOne = puppyRaffle.getActivePlayerIndex(playerOne);
vm.prank(playerOne);
puppyRaffle.refund(index0fPlayerOne);

// We can't select a winner as the contract is insolvent
vm.warp(block.timestamp + duration);
vm.expectRevert("PuppyRaffle: Failed to send prize pool to winner");
puppyRaffle.selectWinner();

// We assert that the total amount collected is greater than the contract balance
// This means that the contract is insolvent
uint256 totalAmountCollected = players.length * puppyRaffle.entranceFee();
assertGt(totalAmountCollected, address(puppyRaffle).balance);
}

```

Notice: Sometimes the `selectWinner` function may work even if the contract is insolvent. This is because of the fees layout. The winner will receive his prize but the fee receiver will never be able to claim his fees, because of this require (the error string message will be irrelevant in such cases):

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are current
```

RECOMMENDED MITIGATION

Track the zero addresses inside of the players array. Make these changes inside of the `selectWinner` function:

```

+ uint256 zeroAddressCount = 0;
+ for (uint256 i = 0; i < players.length; i++) {
+     if (players[i] == address(0)) {
+         zeroAddressCount++;
+     }
+ }
- uint256 totalAmountCollected = players.length * entranceFee;
+ uint256 totalAmountCollected = (players.length - zeroAddressCount) * entranceFee

```

[H-2] Refunding a user zeroes its address in the `players` array, causing a revert on all new entries when two or more users have refunded

HIGH

Fixed

DESCRIPTION

When a user is asking for a refund of his ticket, the `refund` function replaces its address by the zero address in the array, to remove his entry from the raffle. Thus, if multiple users ask for a refund in the same raffle, we will have multiple zero addresses in the array. However, when another user tries to participate to the raffle by calling the `enterRaffle` function, the contract is checking for any duplicate in the array, with this nested loop:

```
for (uint256 i = 0; i < players.length - 1; i++) {  
    for (uint256 j = i + 1; j < players.length; j++) {  
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");  
    }  
}
```

This loop will always revert because of the multiple zero addresses inside of the `players` array, leading to 2 major issues:

1. Users will not be able to enter a raffle where 2 players asked for a refund, causing a DoS until the `players` array is cleaned through the `selectWinner` function.
2. If two participants the raffle ask for a refund before it reaches 4 players, the `selectWinner` will never be callable as needs 4 players or more to be called. The remaining player will have to call the refund function to get his funds back, and the contract will be permanently unusable.

IMPACT

Denial of service on new raffle entries. In the worst case the contract is permanently unusable.

PROOF OF CONCEPT

Actors:

- Players: Normal players entering the raffle.
- Attackers: Malicious players entering the raffle to create a DoS.

```
function test_RaffleDoS() public {  
    // First participant is entering the raffle  
    address[] memory player_1 = new address[](1);  
    player_1[0] = playerOne;  
    puppyRaffle.enterRaffle{value: entranceFee}(player_1);  
  
    // A group of two malicious participants are entering the raffle
```

```

address[] memory attackers = new address[](2);
attackers[0] = attackerOne;
attackers[1] = attackerTwo;
puppyRaffle.enterRaffle{value: entranceFee * 2}(attackers);

// First attacker is refunding his ticket
uint256 indexOfAttackerOne = puppyRaffle.getActivePlayerIndex(attackerOne);
vm.prank(attackerOne);
puppyRaffle.refund(indexOfAttackerOne);

// Second attacker is refunding his ticket too
uint256 indexOfAttackerTwo = puppyRaffle.getActivePlayerIndex(attackerTwo);
vm.prank(attackerTwo);
puppyRaffle.refund(indexOfAttackerTwo);

// Another participant tries to enter the raffle,
// but it will revert because of the duplicate zero addresses
address[] memory player_2 = new address[](1);
player_2[0] = playerTwo;
vm.expectRevert("PuppyRaffle: Duplicate player");
puppyRaffle.enterRaffle{value: entranceFee}(player_2);

// Now there are 3 players in the raffle, and no new participant can enter the raffle
// This does mean that we will never be able to call the selectWinner function
vm.warp(block.timestamp + duration);
vm.expectRevert("PuppyRaffle: Need at least 4 players");
puppyRaffle.selectWinner();

// Now the only way for playerOne to get his funds back is to call the refund function
uint256 indexOfPlayerOne = puppyRaffle.getActivePlayerIndex(playerOne);
vm.prank(playerOne);
puppyRaffle.refund(indexOfPlayerOne);
}

```

RECOMMENDED MITIGATION

To mitigate this critical vulnerability, we advice the following changes on the `enterRaffle` function:

```

for (uint256 i = 0; i < players.length - 1; i++) {
+   if (players[i] == address(0)) continue;
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
}

```

This will avoid checking for duplicates on zero addresses.

[H-3] The `withdrawFees` function may always revert due to integer truncation dust, permanently locking protocol fees

HIGH

Fixed

DESCRIPTION

When a user tries to call the `withdrawFees` function, the contract will go through a require trying to check if there are any active players. To achieve that, it checks if the remaining balance of the contract is strictly equal to the `totalFees` variable. But if we check how this variable is constructed, we can see that it's incremented on every raffle inside the `selectWinner` function:

```
totalFees = totalFees + uint64(fee);
```

And if we check the value of fee, we get this:

```
uint256 fee = (totalAmountCollected * 20) / 100;
```

This line may cause critical issues to the fees withdrawals, because it's a mathematical formula that may truncate its result. In fact the `totalAmountCollected` variable is multiplied by a decimal number (1/5). This does mean that if `totalAmountCollected` is not a multiple of 5, the resulting fee will be truncated by the EVM. It is the same situation for the `prizePool` formula, it may be truncated. Now the variables will always be lower than or equal to the actual amount on Ether sent to the contract. This mean that there will always be some remaining dust and this dust will make the `withdrawFees` function revert everytime on the first require.

IMPACT

Protocol fees are permanently locked in the contract, making the fee receiver unable to claim their revenue.

PROOF OF CONCEPT

To achieve that, we just need to change the `entranceFee` of the raffle contract. Right now the tests are successful because the fee is set to 1e18 which is leading to multiples of 5 in most cases. But if we set the variable to this value, the tests will now fail:

```
uint256 entranceFee = 1e18 + 1;
```

RECOMMENDED MITIGATION

To mitigate this critical vulnerability, we can track if there are active users with another method. Instead of checking if the balance is strictly equal to the claimable fees, we can verify if the `players` array length is 0:

```
function withdrawFees() external {  
-   require(address(this).balance == uint256(totalFees),  
+   require(players.length == 0,  
    "PuppyRaffle: There are currently players active!");  
    uint256 feesToWithdraw = totalFees;  
    totalFees = 0;  
    (bool success,) = feeAddress.call{value: feesToWithdraw}("");  
    require(success, "PuppyRaffle: Failed to withdraw fees");  
}
```

[H-4] The `refund` function is not following the Checks-Effects-Interactions pattern, leading to reentrancy vulnerability

HIGH

Fixed

DESCRIPTION

The `refund` function is sending Ether to `msg.sender` before setting its address to zero in the `players` array. However, this is the only condition used by the `refund` function to check if a user already got refunded:

```
require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not a player");
```

Now an attacker can use a malicious contract to call the `refund` function multiple times through a fallback function, letting him drain all the contract balance with reentrancy.

IMPACT

An attacker can drain the entire contract balance via reentrancy.

PROOF OF CONCEPT

Actors:

- Attacker: Malicious players deploying a contract to drain contract funds.

```
function test_RaffleReentrancy() public playersEntered {
    uint256 balanceBefore = attackerOne.balance;

    // The attacker is deploying a malicious contract
    // that will call the refund function multiple times
    vm.startPrank(attackerOne);
    MaliciousContract maliciousContract = new MaliciousContract(puppyRaffle);
    vm.deal(address(maliciousContract), entranceFee);

    maliciousContract.startAttack();
    maliciousContract.withdraw();
    vm.stopPrank();

    uint256 balanceAfter = attackerOne.balance;
    assertEq(balanceAfter, balanceBefore + 5 * entranceFee);
}
```

RECOMMENDED MITIGATION

To mitigate this critical vulnerability, we need to respect the Checks-Effects-Interactions pattern. Inside of the `refund` function make these changes:

```
- payable(msg.sender).sendValue(entranceFee);
+ players[playerIndex] = address(0);

- players[playerIndex] = address(0);
+ payable(msg.sender).sendValue(entranceFee);
```

[H-6] Overflow can cause a permanent loss of fees

HIGH

Fixed

DESCRIPTION

Everytime the `selectWinner` function is called, the `totalFees` variable is incremented:

```
uint256 fee = (totalAmountCollected * 20) / 100;
totalFees = totalFees + uint64(fee);
```

The `fee` variable is an `uint256` casted to an `uint64`. This cast is not safe because the value can be higher than 2^{64} , which would lead to an overflow. Now everytime the `totalFees` exceed 2^{64} , it will reset to 0, causing a permanent funds loss.

PROOF OF CONCEPT

Actors:

- Players: Normal players entering the raffle.

```
function test_FeeOverflow() public {
    // We deploy a new contract with high entrance fee
    uint256 hugeEntranceFee = 100 * 1e18;
    puppyRaffle = new PuppyRaffle(
        hugeEntranceFee,
        feeAddress,
        duration
    );

    // 4 players enter the raffle
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: 4 * hugeEntranceFee}(players);
    vm.warp(block.timestamp + duration);

    // Now we select a winner and log totalFees
    puppyRaffle.selectWinner();
    uint totalFees = puppyRaffle.totalFees();
    uint expectedFees = (4 * hugeEntranceFee * 20) / 100;

    // `totalFees` is way less than expected fees.
    assertLt(totalFees, expectedFees);
}
```

RECOMMENDED MITIGATION

We advice the protocol to use a newer version of Solidity to handle overflows and revert. An even better solution would be remove the cast to `uint64` as it is useless.

Medium Findings

[M-1] Poor RNG implementation can be exploited by attackers to choose the winner

MEDIUM

Pending

DESCRIPTION

The `selectWinner` function implements a RNG for the `winnerIndex` and for the `rarity`, using the hash of 3 global variable:

```
uint256 winnerIndex =  
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty,  
    uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender, block.difficulty)))
```

These numbers can easily be predicted by anyone before the transaction to be broadcasted, leading to a major vulnerability where malicious actors can call the `selectWinner` function only if they will be selected as a winner. They can also wait for the `rarity` variable to be higher than `LEGENDARY_RARITY`, in order to get minted a rare NFT.

This vulnerability is marked as Medium and not High because it is very unlikely in cases where there a lot of players taking part into the raffle. In fact, the `selectWinner` will be probably called right after the end of `raffleDuration`, and malicious actors will not have the time to wait for `block.timestamp` and `block.difficulty` to match his needs.

However `block.difficulty` can probably be manipulated by Ethereum validators and in this case it would be a critical vulnerability.

IMPACT

Malicious actors can manipulate raffle outcomes to guarantee winning and farm rare NFTs.

PROOF OF CONCEPT

Actors:

- Attackers: Malicious users exploiting the poor RNG.

```
function test_RafflePoorRNG() public playersEntered {  
    // A group of two malicious participants are entering the raffle  
    address[] memory attackers = new address[](2);  
    attackers[0] = attackerOne;  
    attackers[1] = attackerTwo;
```

```

// Save attackers' balance before they enter the raffle
vm.deal(attackerOne, entranceFee * 2);
uint256 balanceBefore = attackerOne.balance + attackerTwo.balance;

// Attackers enter the raffle
vm.prank(attackerOne);
puppyRaffle.enterRaffle{value: entranceFee * 2}(attackers);

// Raffle duration ends
vm.warp(block.timestamp + duration);

// The attacker waits for the `winnerIndex` to match
// one of the attackers' addresses
uint256 winnerIndex = 0;
uint256 attackerOneIndex = puppyRaffle.getActivePlayerIndex(attackerOne);
uint256 attackerTwoIndex = puppyRaffle.getActivePlayerIndex(attackerTwo);

// Here
while (winnerIndex != attackerOneIndex && winnerIndex != attackerTwoIndex) {
    // Wait next block
    vm.roll(block.number + 1);
    vm.warp(block.timestamp + 8);

    // Calculate new winnerIndex
    winnerIndex = uint256(keccak256(abi.encodePacked(
        attackerOne, block.timestamp, block.difficulty
    ))) % 6;
}

// Now attackers can select the winner because it will match one of their addresses
vm.prank(attackerOne);
puppyRaffle.selectWinner();

// Check that attackers' balance has increased
uint256 balanceAfter = attackerOne.balance + attackerTwo.balance;
assertGt(balanceAfter, balanceBefore);
}

```

RECOMMENDED MITIGATION

To generate a random number, the safest way is to use an external oracle.

[M-2] The total amount of player inside a raffle is limited

MEDIUM

Fixed

DESCRIPTION

When a single users is joining a raffle by calling the `enterRaffle` function, his address is added to the `players` list. This call doesn't cost much gas. However, the duplicates checking loop, will go through all the existing players of the list to check if the new player already exists. For every new player entering the raffle, the cost will be higher, leading to 2 issues:

- The price to enter the raffle will be very high for the last players compared to thee firsts to enter.
- At some point, users will not be able to enter the raffle because the transaction gas cost will be too high. After some tests, we found that the maximum total amount of players that can enter the raffle is .

PROOF OF CONCEPT

Actors:

- Players: Normal players entering the raffle.
- Last Player: Normal player that can not enter the raffle.

```
function test_NoPlayerCanEnter() public {
    // Create a list of 242 players to fill the array (max simultaneously)
    address[] memory players = new address[](242);
    for (uint256 i = 0; i < 242; i++) {
        players[i] = address(uint160(i+1));
    }

    // 242 players enter the raffle simultaneously
    puppyRaffle.enterRaffle{value: players.length * entranceFee}(players);

    // Now 26 players enter the raffle one by one, paying a lot of gas
    for (uint256 i = 0; i < 26; i++) {
        address[] memory newPlayer = new address[](1);
        newPlayer[0] = address(uint160(i+243));

        puppyRaffle.enterRaffle{value: entranceFee}(newPlayer);
    }

    // The 269th player tries to enter the raffle, but the gas cost is too high
    address[] memory lastPlayer = new address[](1);
    lastPlayer[0] = address(uint160(269));

    // We log the gas here
    uint256 gasBefore;
    uint256 gasAfter;
```

```
uint256 usedGas;  
gasBefore = gasleft();  
puppyRaffle.enterRaffle{value: entranceFee}(lastPlayer);  
gasAfter = gasleft();  
  
// The transaction will revert  
usedGas = gasBefore - gasAfter;  
assertGt(usedGas, 30_000_000);  
}
```

RECOMMENDED MITIGATION

We should remove the duplicate check loop. In Ethereum blockchain, people can create as many addresses as they want. So if they want to enter multiple times the raffle, they will still be able to anyway. Another advice would be to use mappings instead of arrays.

Low Findings

[L-1] The amount of players that can enter the raffle simultaneously is limited

LOW

Fixed

DESCRIPTION

When a list of users are joining a raffle by calling the `enterRaffle` function, each player is added to the `players` list one by one with a for loop. The bigger the `newPlayers` array is, the more gas the transaction will use. There is also the duplicates checking loop, this one is a double nested for loop which is exponentially more costly in term of gas. Ethereum blocks have a maximum size of 30 million gas, which means that the transaction will always revert if the players amount is too high. Let's see how many players can enter the raffle. After some tests, we found that the maximum amount of players that can enter the raffle simultaneously is 241.

PROOF OF CONCEPT

Actors:

- Players: Normal players entering the raffle.

```
function test_MaxPlayersToEnter() public {
    // We create a list of 242 players
    address[] memory players = new address[](242);
    for (uint256 i = 0; i < 242; i++) {
        players[i] = address(uint160(i+1));
    }

    // The 242 players try to enter the raffle
    // and we save the gas used for the transaction
    uint256 gasBefore = gasleft();
    puppyRaffle.enterRaffle{value: players.length * entranceFee}(players);
    uint256 gasAfter = gasleft();

    // The transaction will revert
    uint256 usedGas = gasBefore - gasAfter;
    assertGt(usedGas, 30_000_000);
}
```

RECOMMENDED MITIGATION

We advice the protocol to limit the array size in the frontend to avoid transations reverting. It should also be specified in the docs. If people want to enter more than 242 players, they should call multiple transactions, but they will still be limited at some point (see M-2).

Informational Findings

[I-1] The `\_isActivePlayer` internal function is never used

INFO

Acknowledged

DESCRIPTION

This is consuming gas by increasing compiled code size.

RECOMMENDED MITIGATION

Delete this function.